



第二章 Python基础知识

赵晓航

xiaohangzhao@mail.shufe.edu.cn

上海财经大学·信息管理与工程学院



目录

1. 常量
2. 变量
3. 运算
4. 注释
5. Python程序内部运行过程
6. PEP8编程风格
7. 应用案例
8. 本章习题



1. 常量

- **常量**：指程序执行过程中其值保持不变的元素。
- 常见常量类型包括**数字**和**字符串**
 - **数字**：在Python中分为三种主要类型：
 - **整数 (Integers)**：这是基础的数字类型，包括所有正负整数和零。
 - **浮点数 (Floats)**：代表小数，也可以表示为科学计数法，如6.02E23表示 6.02×10^{23} 。
 - Python中的浮点数通常为64位双精度标准（使用IEEE 754标准），表示范围约为 -1.8×10^{308} 到 1.8×10^{308} ，精度大约为15位小数。然而，浮点数在高精度运算中可能会出现误差，尤其是在处理极小或极大的数值时需要谨慎
 - **复数 (Complex)**：由实数部分和虚数部分构成，格式为 $x + yj$ ，其中x和y为实数
 - **与AI协作探索**：如何查看浮点数的二进制表示？什么是IEEE 754标准？



字符串

- **字符串**：是由字符组成的序列，用于表示文本。在Python中，字符串被定义为**不可变类型**

- 在python程序中，一个字符串必须用**单引号** `' '` 或**双引号** `" "`括起来，字符串中的各类字符，如空格等都保持原样

```
1 'I am studying python'
2 "I am studying python"
```

- 如果是多行字符串，需要使用**三引号**，即`'''`或`"""`，在三引号中，可以使用单引号或双引号

```
1 '''
2 Life is short, use Python.      -- 人生苦短，我用python
3 Life's pathetic, let's pythonic. -- 人生苦短，python是岸
4 '''
```



字符串格式化

- 字符串的格式化:

- **str.format()**: 方法用于创建格式化的字符串, 通过替换字符串中的占位符来工作。
占位符由一对大括号 {} 表示, 可以包含一个可选的索引或关键字, 用于指定对应的 format() 方法参数。
- "Hello, {}. You are {} years old.".format("Alice", 30)
 - 结果: "Hello, Alice. You are 30 years old. "
- print('{0}加上{1}等于{2}'.format(10, 20, 10 + 20))
 - 最终打印的字符串是"10加上20等于30 "



格式化函数Format()

- **format()**方法还支持多种格式化选项

- **1. 控制小数点后的位数**

- `"The number is {:.2f}".format(3.14159)`
- 其中`{:.2f}` 指定了格式化的样式。`.` 表示小数点，`2` 表示小数点后保留两位，而 `f` 表示浮点数。所以 `3.14159` 被格式化为 `3.14`

- **2. 对齐字符串**

- **左对齐:**
 - `"Left aligned: {:<10}".format("test")`
 - `:<10` 表示字符串应该在一个宽度为 10 的字段内左对齐。`"test"` 左对齐，并且右侧填充空格直到总长度达到 10
- **右对齐:**
 - `"Right aligned: {:>10}".format("test")`
 - `:>10` 表示字符串在宽度为10的字段内右对齐。`"test"` 右对齐，并且左侧填充空格
- **居中对齐:**
 - `"Center aligned: {:^10}".format("test")`
 - `:^10` 表示字符串在宽度为 10 的字段内居中对齐。`"test"` 居中，左右两侧均匀填充空格



格式化函数Format()

3. 填充和宽度:

- `"Padded: {:*^10}".format("test")`
- `:*^10` 表示字符串在宽度为 10 的字段内居中对齐，并用 * 进行填充。因此，"test" 被 * 符号包围，直到总长度达到 10

4. 数字格式化:

- 整数的千位分隔符: `"Number with commas: {:,}".format(123456789)`
:, 表示在数字中添加千位分隔符。因此，123456789 被格式化为 123,456,789
- 以百分比形式表示: `"Percentage: {:.2%}".format(0.756)`
:.2% 将数字转换为百分比形式，并保留两位小数。因此，0.756 被转换为 75.60%

5. 格式化日期:

- `import datetime`
`now = datetime.datetime.now()`
`"Current date: {%Y-%m-%d %H:%M}".format(now)`
- `{:%Y-%m-%d %H:%M}` 定义了日期和时间的格式。%Y 表示四位年份，%m 表示月份，%d 表示日，%H 表示小时，%M 表示分钟
- **思考:** 怎么将月份信息用英文缩写展示出来?



字符串中的特殊字符

- 在字符串中，某些特殊字符具有特定的作用

特殊字符	含义	代码举例	运行结果
\n	换行符，使光标移到下一行的开头	print('python\nJava')	Python Java
\t	水平制表符，常用于添加水平空间，类似按下Tab键	print('python\tJava')	python Java
\	转义符(escape character)，用于插入那些通常会被解释为特殊字符的字符，如引号、\t等。在这些字符前添加\，可以将它们转换为普通字符，从而正常显示。	print('python\'s book')	python's book

- 在有大量特殊字符的情况下，为了避免反复使用转义符，可以将字符串定义为原始字符串。可以通过在字符串前添加 **r** 或 **R** 来实现
 - `print(r"python's book.\n")`
 - \n 不会被解释为换行符，而是作为普通的反斜杠和字符 n 显示



2. 变量

- **变量**：计算机编程中一个基本概念，代表着**可变的数据**
 - 变量的值在程序运行过程中可以改变
 - 本质上，**变量是对存储数据值的内存空间的命名**，变量名指代该内存空间或其首地址，存储在其中的数据值则会随着程序的运行而变化
- **变量名称**：指向内存中存储的数据值的标识符，其命名应遵循以下规则：
 - **名称的首字符**必须是字母（A-Z或a-z）或下划线（_）
 - **后续字符**可以是字母、数字（0-9）或下划线
 - **变量名区分大小写**，例如，age和Age会被视为不同的变量
 - 变量命名一般使用有意义的名称，以提高代码的可读性
 - 例如，使用customerName或customer_name等，通过**驼峰命名法**或**下划线**来连接多个单词



变量的声明

- 变量在首次赋值时声明，其类型由赋值时的数据类型决定
- Python是一种动态类型语言，这意味着同一个变量在程序运行期间可以被赋予不同类型的值。
 - `name = 'Edward'`
`age = 10`
`age = 10.5`
 - 其中，变量name被声明为字符串，而age先是整型后变为浮点型

【变量的类型】

虽然变量的类型可以随时改变，但在实际编程中，保持变量类型的一致性有助于代码的可读性和维护性，提高程序的整体稳定性



Python的保留字

- **Python保留字**不能用作变量名，这些保留字在语言中有特殊的意义。

Python 3中的保留字：

分类	保留字
逻辑运算	and, or, not
条件控制	if, else, elif, while, for, break, continue
异常处理	try, except, finally, raise
定义结构	def, class, lambda
导入模块	import, from
变量作用域	global, nonlocal
特殊值	True, False, None
其他	pass, del, assert, with, yield, as, return



变量赋值

- **变量赋值：**

- 使用等号 `=` 给变量赋值，即将等号右边的表达式的值赋予等号左边的变量。
- **单变量赋值语法**
 - `a_variable = a_expression`
 - 其中，`a_variable` 代表一个变量，而 `a_expression` 代表一个表达式。这个表达式可以是一个常量、另一个变量、两者之间的运算，甚至可以是一个函数调用
- **多变量赋值语法**
 - `variable_1, ..., variable_N = expression_1, ..., expression_N`
 - 等号右边的表达式 `expression_1, ..., expression_N` 的值依次赋给左边的变量 `variable_1, ..., variable_N`
 - 这种多变量赋值通常用于逻辑上相关联的变量，可以一起批量处理



多变量赋值举例

- **题目：**假设需要将平面坐标系上的点 $(x1, y1)$ 沿水平方向向右移动 δx ，沿垂直方向向下移动 δy ，计算该点的新坐标值。

- 设初始状态下， $x1$ 和 $y1$ 的值分别为1和2， δx 和 δy 的值分别为5和10

`x1, y1 = 1, 2`

`delta_x, delta_y = 5, 10`

`x1, y1 = x1 + delta_x, y1 - delta_y`

`print(x1, y1)`



变量作用域

- **变量作用域：**变量只能在声明（或定义）后的特定区域内被访问或修改，这个区域称为变量的作用域。
- 作用域取决于变量的声明位置及程序中访问该变量的范围。通常，变量的作用域分为局部作用域和全局作用域：
 - **局部作用域内的变量：**只能在定义它的函数或代码块中访问
 - **全局作用域内的变量：**则可以在整个程序中访问
- 不同作用域中的同名变量不会相互影响，遵循“**就近原则**”来决定优先级



数据类型

- Python中的变量可以是多种数据类型，包括数字、字符串和四种内嵌的集合类型：**列表 (list)**、**元组 (tuple)**、**字典 (dict)** 和**集合 (set)**。此外，Python支持面向对象编程，允许用户**自定义数据类型**

类别	数据类型	描述	示例
数字类型	int	整数	5, -3, 42
	float	浮点数	3.14, -0.001
	complex	复数	3 + 4j
文本类型	str	字符串	"hello", 'Python'
序列类型	list	有序、可变的集合	[1, 2, 3]
	tuple	有序、不可变的集合	(1, 2, 3)
映射类型	dict	键值对集合	{'name': 'Alice', 'age': 25}
集合类型	set	无序、不重复的元素集	{1, 2, 3}
	frozenset	不可变的集合	frozenset({1, 2, 3})
布尔类型	bool	布尔值	True, False
二进制类型	bytes	不可变的字节序列	b'hello'
	bytearray	可变的字节序列	bytearray(b'hello')
	memoryview	内存视图	memoryview(bytes(5))
None类型	NoneType	表示无值或空	None



3. 运算

- **运算**：通过**运算符**将常量或变量关联起来形成表达式
 - 例如， $1 + 2$ 是一个简单的表达式。在表达式中，常量和变量被称为**操作数** (operands)，例如1和2，而**运算符**则用于执行类似加减乘除的操作，例如+。
- **内置数值运算符**
 - Python提供了一系列基本的数值运算符，用于执行常见的数学运算。

运算符	描述	例子
+ (加法)	两个对象相加	1+2 输出3
- (减法)	从一个数中减去另一个数	100-20输出80
* (乘法)	两个数相乘或返回字符串重复指定次数的结果	2 * 5 输出10 bala * 2输出balabala
** (乘方)	一个数的另一个数次方	2 ** 8 输出256
/ (除法)	一个数除以另一个数	1/2 输出 0.5
// (整除)	一个数除以另一个数的结果向下取整	8 // 3 输出 2
% (模运算)	除法运算后的余数	8 % 5 输出 3
<< (左移)	将数字的二进制位向左移动指定的位数	3 << 2 输出 12
>> (右移)	将数字的二进制位向右移动指定的位数	13 >> 2 输出3
& (按位与)	对数字进行按位与操作	7 & 3 输出 3
(按位或)	对数字进行按位或操作	10 4 输出 14
^ (按位异或)	对数字进行按位异或操作	5 ^ 3 输出6
~ (按位取反)	对数字进行按位取反操作	~5 输出 -6



运算符

- 布尔逻辑运算符：于生成布尔值（True或False）

运算符	描述	例子
<（小于）	返回x是否小于y，结果为True或False (首字母大写)	6 < 4 输出False
>（大于）	返回x是否大于y	3 > 2 输出True
<=（小于等于）	返回x是否小于或等于y	2 <= 3 返回True
>=（大于等于）	返回x是否大于或等于y	2 >= 3 返回False
==（等于）	比较两个对象是否相等	3 == 3 返回True
!=（不等于）	比较两个对象是否不相等	1 != 2 返回True
and（与）	如果x是False，则返回False，否则返回y的计算值	False and True 返回False
or（或）	如果x是True，则返回True，否则返回y的计算值	False or True 返回True

- 与AI协作探索：表达式 “1 * False” 与 “1 + True” 分别等于什么？逻辑运算符可否用代数运算符等价表达？有什么相关应用？



内置的标识运算符

- **内置的标识运算符**：在 Python 中，**is** 被归类为**标识运算符** (Identity Operator) 。标识运算符是用来比较两个对象的内存地址，从而判断它们是否为同一个对象
- **标识运算符有两个**：
 - **is**：如果两个变量指向内存中的同一个对象，则返回 True
 - **is not**：如果两个变量指向不同的对象，则返回 True
 - 该类运算符通常用于检查一个变量是否与特定的对象相同
 - `a = None`
`if a is None:`
`print("a is None")`
 - 注意，None 是一个单例，所有指向 None 的引用都指向内存中的同一个对象



将 is (或 is not) 与 == (或 !=)

- 不要将 is (或 is not) 与 == (或 !=) 混淆
- == 运算符：用于比较两个变量的值是否相等
- is 运算符则：用来比较两个变量是否为同一个对象
 - 对于小的整数或字符串（由于 Python 对这些小的、常用的对象进行了内存优化），is 和 == 可能会表现出相同的结果：
a = 10
b = 10
print(a is b) # 通常会输出 True, 因为小整数对象被重用
print(a == b) # 输出 True, 因为它们的值相等
 - 但对于大的整数或更复杂的对象，is 和 == 表现会不同：
a = 1000
b = 1000
print(a is b) # 通常会输出 False, 因为它们是不同的对象
print(a == b) # 输出 True, 因为它们的值相等
 - 在 if a is not b 语句中，检查 a 和 b 是否不是同一个对象。例如：
a = [1, 2, 3]
b = [1, 2, 3]
if a is not b:
 print("a and b are not the same object")
 - 即使 a 和 b 的内容相同，但二者是内存中的两个不同的列表对象。
 - 虽然语句 if a is not b 与 if not a is b 运行效果相同，但 PEP 8 编程风格规范依然建议采用第一种行内否定方式



内置的数值运算函数

- **内置的数值运算函数：**Python解释器内嵌了一系列用于数值运算的函数，为编程提供了基本的数学运算能力

函数	描述	例子
abs(x)	返回x的绝对值	abs(-10) 输出 10
pow(x, y)	计算x的y次幂	pow(10, 2) 输出 100
round(x, ndigits)	将x四舍五入到小数点后ndigits位	round(3.1415, 2) 输出 3.14
divmod(x, y)	同时返回x除以y的商和余数	divmod(21, 5) 输出 (4, 1)
max(x1,...,xn)	返回x1,..., xn中的最大值	max(2, 20, 8) 输出 20
min(x1,...,xn)	返回x1,..., xn中的最小值	min(2, 20, 8) 输出 2

- **与AI协作探索：**round函数采用“银行家舍入法(Bankers Rounding)”，又称“四舍六入五成双”
 - 理论上讲，round(3.145, 2) 应得到3.14，而 round(1.25, 1) 应得到1.2
 - 但实际结果是什么？为何与理论结果不同？



应用实例

- **题目：**假设2000年初，个人A在银行存入1万元，年利率为2%；同时，个人B投资1万元于年利率为10%的基金。计算到2030年初，两人各自的本息总额
 - `account_a, account_b = 10000, 10000`
`interest_rate_a, interest_rate_b = 0.02, 0.1`
`total_a = account_a * pow((1 + interest_rate_a), 30)`
`total_b = account_b * pow((1 + interest_rate_b), 30)`
`print("Total asset of A: ",round(total_a))`
`print("Total asset of B: ",round(total_b))`
 - 输出结果：
Total asset of A: 18114
Total asset of B: 174494
- **思考：**如果不进行红利再投，本息总额应该如何计算？



内置的数值转换函数

- **内置的数值转换函数：**利用数值类型转换函数，将一种数值类型转化为另一种类型

函数	描述	例子
int(x)	将x转换为整型	int(3.14)输出3， int('5')输出5
float(x)	将x转换为浮点型	float(5)输出5.0， float('8.02')输出8.02
complex(re, im)	生成由实部re和虚部im构成的复数	complex(2, 3)输出2+3j



math库

- **math库**：Python的标准库中包含了一系列强大的函数库，其中math库是专门用于数学运算的内置库。这些库可分为两类：
 - **标准库**：这些是Python环境中默认包含的库，无需额外安装即可使用
 - **第三方库**：不包含在标准Python安装中，需要单独安装才能使用的库
- math库提供了一系列用于执行数学计算的函数和常数，包括基本的数学常数（如 π 和 e ）以及多种数值计算函数
- math库包括以下几类功能：
 - **数学常数**：如 π (pi) 和自然对数的底数 e
 - **基本数学运算**：如平方根 (sqrt)、三角函数 (sin、cos、tan等) 和对数函数 (log、log10)
 - **高级数学运算**：如幂函数 (exp)、向上和向下取整 (ceil、floor) 等



常用math库函数

函数或常数	数学表示	描述
math.pi	π	圆周率 π 的近似值
math.e	e	自然对数的底数 e 的近似值
math.sqrt(x)	$\sqrt{x}$	计算x的平方根
math.sin(x)	$\sin(x)$	计算x的正弦值
math.cos(x)	$\cos(x)$	计算x的余弦值
math.tan(x)	$\tan(x)$	计算x的正切值
math.asin(x)	$\arcsin(x)$	计算x的反正弦值
math.acos(x)	$\arccos(x)$	计算x的反余弦值
math.atan(x)	$\arctan(x)$	计算x的反正切值
math.log(x)	$\ln(x)$	计算x的自然对数（以e为底）
math.log10(x)	$\log_{10}(x)$	计算x的以10为底的对数
math.exp(x)	e^x	计算e的x次幂
math.ceil(x)	$\lceil x \rceil$	返回大于或等于x的最小整数
math.floor(x)	$\lfloor x \rfloor$	返回小于或等于x的最大整数

• 在使用math库的函数之前，可以采用以下方法导入特定的函数：

• **from math import function_name**

- 其中function_name是想使用的特定函数
- 例如，导入并使用sqrt()函数
- **from math import sqrt**
- **result = sqrt(25)**
- **print(result) # 输出: 5.0**



4. 注释

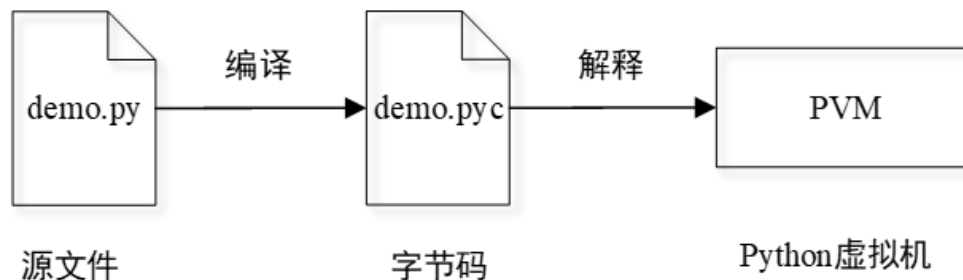
- **注释**是用来对代码进行解释性说明的文本，它们不仅有助于程序员理解代码的功能和目的，也使代码更易于维护。**注释是为人类阅读而编写的，而非计算机执行**
- **单行注释**：使用#符号进行标记。#后面的文本会被视为注释
 - `name = 'python' # 定义名为name的变量，存储字符串'python'`
`# 下面这行代码将打印变量name的内容`
`print(name)`
- **多行注释**：使用一对三引号（""" 或 '''）包围的文本块来创建
 - `"""`
这段代码演示了一个简单的打印功能。
它会输出字符串'hello world'。
`"""`
`print('hello world')`



5. Python程序内部运行过程

- **Python程序运行：**

1. **编写程序：**编写Python代码，并将其保存为.py格式的文件，如demo.py。
2. **编译成字节码：**当首次运行Python程序时，Python解释器将源代码（.py文件）编译成字节码。字节码是一种介于源代码和机器代码之间的低级代码形式，具有平台无关性。该字节码通常保存在以.pyc为扩展名的文件中，存储于磁盘上。字节码的使用旨在提高程序运行的速度。
3. **解释执行：**编译后的字节码被送到Python虚拟机（PVM, Python Virtual Machine）上进行解释和执行。Python虚拟机是Python解释器的一部分，负责读取字节码并执行相应的操作。
4. **重复执行优化：**如果相同的Python程序需要再次运行，Python解释器会优先使用已生成的.pyc文件中的字节码，而非重新从源代码编译。这一步骤进一步提高了程序执行的效率





6. PEP 8 编程风格

- Python社区制定了一套代码风格指南，即**PEP 8** (Python Enhancement Proposal #8)
- PEP 8 包含了多方面的编程风格规范，涵盖了代码布局、引号使用、命名规则、空格运用、表达式书写等多个方面
 - **缩进**：使用空格进行缩进，每次缩进由4个空格组成，不使用制表符（Tab键）进行缩进
 - **行宽限制**：每行代码长度不应超过79个字符，这样做旨在提升代码的可读性和便于维护
 - **变量赋值**：在赋值符号（=）的左右两侧各添加一个空格，以增强代码的清晰度
 - **变量类型注解**：变量名与冒号之间不加空格，冒号与类型信息之间加一个空格
`age: int # 变量名与冒号之间无空格，冒号与类型信息之间有一个空格`
 - **变量命名**：变量名称应使用小写字母，若由多个单词组成，则用下划线（_）连接。这种命名方式被称为蛇形命名法（snake_case），它有助于提高代码的可读性，使变量名更加易于理解

```
user_name = "Alice"
```



7. 应用案例1：货币汇率转换

- **需求描述：**某公司经常需要进行外汇交易，特别是将美元转换为人民币。为此，他们需要一个简单的程序来执行这项转换。

- # 变量的作用域: USD_TO_RMB是一个全局常数

USD_TO_RMB = 6.5 # 数字: 这是一个常数, 表示当前的汇率

字符串: 使用字符串提示用户

usd_amount = float(input("请输入要转换的美元数量: ")) # 数字: 数据类型为浮点数

内置数值运算符: 进行基本的数学运算

rmb_amount = usd_amount * USD_TO_RMB

输出结果

print("{} 美元等于 {:.2f} 人民币".format(usd_amount, rmb_amount))



从“汇率转换”见大国崛起

- **汇率：不只是数字，更是国家实力的“晴雨表”**

- **经济强，则货币强：** 汇率稳定攀升，反映的是我国强大的经济基础、制造能力和国际市场信心。
- **影响力：** 中国作为全球最大贸易国之一，人民币的波动牵动世界神经

- **科技：人民币国际化的“加速器”**

- **传统痛点：** 跨境支付手续繁、成本高、速度慢
- **时代变革：** 传统金融体系效率瓶颈凸显，区块链等新技术正重塑全球支付格局
- **中国方案：**
 - **数字人民币 (e-CNY)：** 打破壁垒，实现支付即结算，提升跨境效率与安全性
 - **跨境支付系统 (CIPS)：** 是中国建立的、专门用于处理人民币跨境支付业务的清算系统，它减少了对西方主导的SWIFT系统的依赖，是推动人民币国际化、保障国家金融安全的关键支柱。金融科技的“自主路”，减少对国外系统的依赖，保障金融安全

- **结论：科技自立自强，是让人民币走向世界、赢得话语权的硬核支撑！**





案例2：简单的财务分析

- **需求描述：**公司希望能够快速评估其年度财务表现。需要输入去年和今年的销售额和成本，计算出利润率和销售额增长率。

- *# 获取去年和今年的销售额及成本*

```
sales_last_year = float(input("请输入去年的销售额: "))
```

```
sales_this_year = float(input("请输入今年的销售额: "))
```

```
cost_last_year = float(input("请输入去年的成本: "))
```

```
cost_this_year = float(input("请输入今年的成本: "))
```

- # 计算去年和今年的利润*

```
profit_last_year = sales_last_year - cost_last_year
```

```
profit_this_year = sales_this_year - cost_this_year
```

- # 计算利润率和销售额增长率*

```
profit_margin_last_year = (profit_last_year / sales_last_year) * 100
```

```
profit_margin_this_year = (profit_this_year / sales_this_year) * 100
```

```
growth_rate = ((sales_this_year - sales_last_year) / sales_last_year) * 100
```

- # 输出财务分析结果*

```
print("去年的利润率为: {:.2f}%".format(profit_margin_last_year))
```

```
print("今年的利润率为: {:.2f}%".format(profit_margin_this_year))
```

```
print("销售额增长率为: {:.2f}%".format(growth_rate))
```

- **思考：**如何计算利润率的增长率？



案例3：放射性同位素衰减

- 科学家通过对原子核研究发现，所有放射性物质其原子核数目随时间作负指数函数而衰减，称为**衰变定律**。该定律使用下列公式计算，经过一定时间后放射性元素的剩余量：

$$N_t = N_0 e^{-0.693t/T}$$

其中， N_0 是放射性物质的初始质量， N_t 是经过时间 t 之后仍然剩余的质量， T 是该物质的半衰期。

现有同位素锶-90的初始质量为100mg，半衰期为38.1年，求200年后的剩余质量

- `from math import exp # 导入指数函数`
- `# 锶-90的半衰期、初始质量和时间设定`
- `half_life = 38.1`
- `initial_mass = 100`
- `years = 200`
- `# 计算锶-90经过200年后的剩余质量`
- `remaining_mass = initial_mass * exp(-0.693 * years / half_life)`
- `print('Remaining mass of Sr-90:', remaining_mass)`



本章习题

• 一. 理论知识类习题

1. 解释常量的概念，并列举Python中常量的主要数据类型。
2. 阐述变量在Python程序中的作用，并说明变量命名应遵循的规则。
3. 分析以下情景：如果在文中“内置数值运算函数”的示例中，不先为变量account_a、account_b、interest_rate_a和interest_rate_b赋初值，而是直接在计算表达式中使用相应的数值，结果会有什么不同？比较这两种方法的优缺点。
4. 描述Python中字符串的不可变性是什么意思，并举例说明。
5. 阐述Python中的逻辑运算符，并通过实例说明其在条件判断中的应用。
6. 在Python中，解释is和 == 的区别，并通过示例代码展示它们的不同应用场景。



• 二. 编程实践

1. 编写程序计算并输出半径为10cm的球体的表面积和体积。
2. 编写程序计算并显示任意两个数字的最大公约数（GCD）。（提示：可使用math库中 gcd() 函数）
3. 编写程序计算并输出给定数字的阶乘。
4. 假设拥有20万元，面临两种选择：一是投资一个年化受益率为15%的实业项目，二是购买一辆每年折旧率为15%的小汽车。请编写一个Python程序，计算并比较10年后这两种选择的资产值。
5. 利用Python中的math库，编写程序计算并输出给定角度（以度为单位）的正弦、余弦和正切值。
6. 编写一个Python程序，要求用户输入三个单词（例如：苹果、香蕉、樱桃），然后使用字符串的 format()方法按列对齐这些单词。第一个单词右对齐，第二个单词居中，第三个单词左对齐，每列宽度固定为10个字符。
7. 编写一个Python程序，显示当前日期和时间，并格式化输出为"年-月-日 时:分:秒"的形式。（提示：使用datetime模块获取当前日期和时间）



The End !